

RISC-V-based AES Accelerator for Wearable Devices

Milan Patel, Buddhi Perera, and Johnson Yang

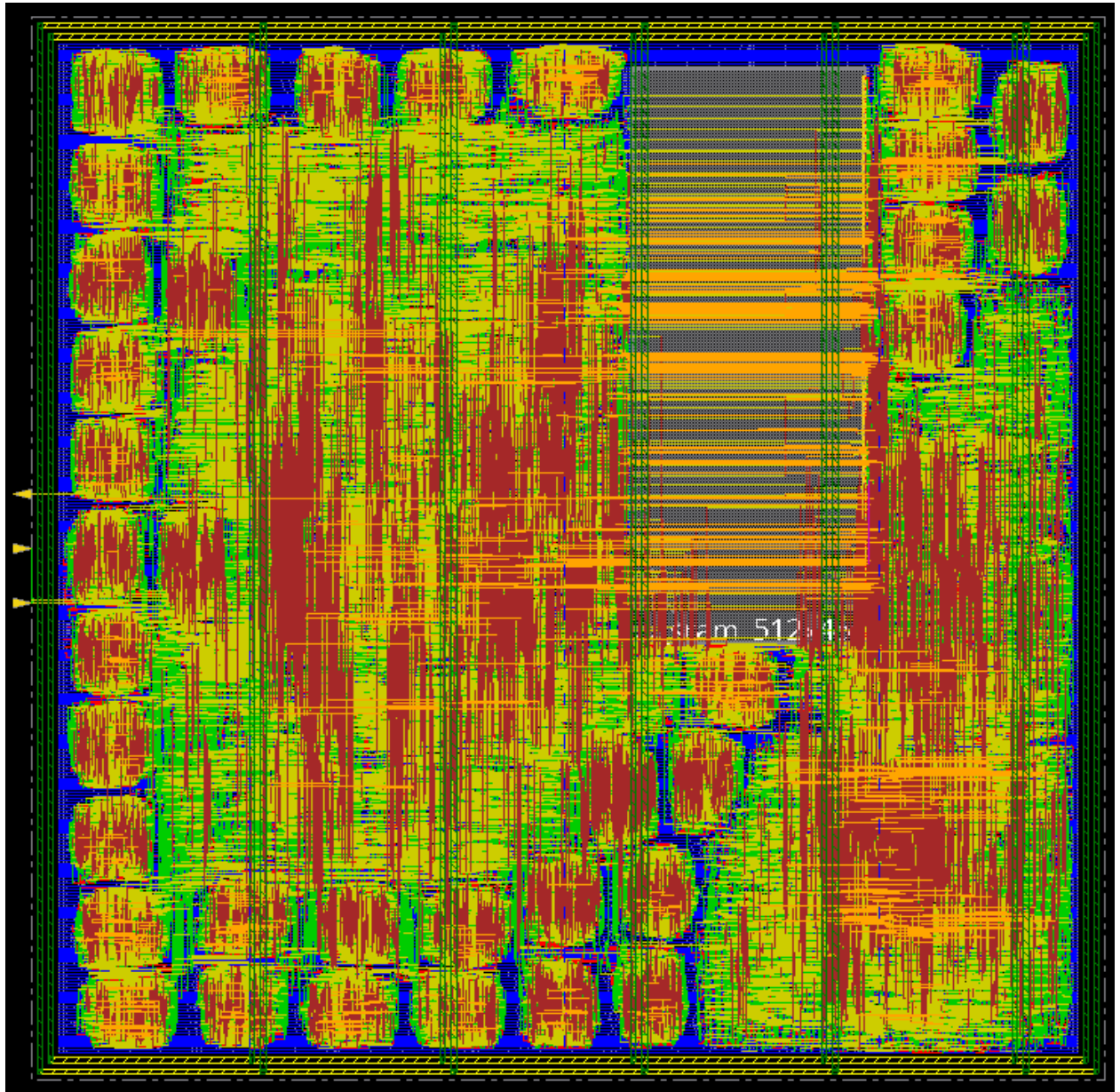


Figure 1. Chip layout through Innovus GUI depicting a SRAM macro on the upper right, and standard cells used in the netlist of the AES module and PicoRV32 core. The surrounding power ring uses vertical layer m11 and horizontal layer m10, power stripes provide power/ground to our macro and routes on each row do the same for our standard cells, fillers, and taps.

```
#####
# Generated by:      Cadence Innovus 23.14-s088_1
# OS:               Linux x86_64(Host ID ecs03.poly.edu)
# Generated on:      Fri Dec 12 19:54:25 2025
# Design:           system_top
# Command:          time_design -timing_debug_report -sign_off -
path_report -drv_report -slack_report -num_paths 50 -report_prefix
system_top_signOff_setup -report_dir timingReportsFinal
#####
```

OptSummary:

time_design Summary

Setup mode	all	reg2reg	default
WNS (ns):	4.439	4.439	8.185
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	3749	3268	3037

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 43.993%

(100.000% with Fillers)

Total number of glitch violations: 0

Figure 2. Setup timing report indicating our chip has no violating paths and passes design rule violation checks. This report uses the sign-off option of the time_design command, with the command shown at the top.

```
#####
# Generated by:      Cadence Innovus 23.14-s088_1
# OS:               Linux x86_64(Host ID ecs03.poly.edu)
# Generated on:      Fri Dec 12 19:55:24 2025
# Design:           system_top
# Command:          time_design -timing_debug_report -sign_off -hold -
path_report -slack_report -num_paths 3000 -report_prefix system_top_signOff
-report_dir timingReportsFinal
#####
```

OptSummary:

time_design Summary				
Hold mode	all	reg2reg	default	
WNS (ns):	0.000	0.006	0.000	
TNS (ns):	0.000	0.000	0.000	
Violating Paths:	0	0	0	
All Paths:	3749	3268	3037	

Density: 43.993%
(100.000% with Fillers)

Figure 3. Hold timing report obtained from Innovus using signoff procedures. We note there are no violating paths in this check.

```
#####
# Generated by:      Cadence Innovus 23.14-s088_1
# OS:               Linux x86_64(Host ID ecs03.poly.edu)
# Generated on:      Fri Dec 12 19:52:04 2025
# Design:           system_top
# Command:          check_drc
#####
#set_verify_drc_mode -area {0 0 0 0} -check_ndr_spacing auto -check_only
default -check_same_via_cell true -disable_rules "" -exclude_pg_net false -
ignore_cell_blockage false -ignore_trial_route false -limit 1000 -report ../
files/system_top.drc.rpt -use_min_spacing_on_block_obs auto
```

No DRC violations were found

Figure 4. DRC report from Innovus, checking the entire chip and enabling all rules. The output can be seen at the bottom indicating there are no DRC violations.

```

=====
=== CALIBRE::DRC-H SUMMARY REPORT
===
Execution Date/Time:      Fri Dec 12 20:20:52 2025
Calibre Version:         v2024.2 29.16   Thu May 2 07:35:42 PDT 2024
Rule File Pathname:      _N16ADFP_DRC_Calibre_11M.11_1a.encrypt_
Rule File Title:
Layout System:           GDS
Layout Path(s):          /home/jy5187/Documents/soc/project/aes-accelerator/physical_design/system_top.gds
Layout Primary Cell:     system_top
Current Directory:        /home/jy5187/Documents/soc/aes-accelerator/physical_design/logs
User Name:                jy5187
Maximum Results/RuleCheck: 1000
Maximum Result Vertices: 4096
DRC Results Database:     /home/jy5187/Documents/soc/project/aes-accelerator/calibre/drc.results (ASCII)
Layout Depth:            ALL
Text Depth:              PRIMARY
Summary Report File:      drc.summary (REPLACE)
Geometry Flagging:        ACUTE = YES  SKEW = YES  ANGLED = NO  OFFGRID = NO
                           NONSIMPLE POLYGON = YES  NONSIMPLE PATH = YES

Excluded Cells:
CheckText Mapping:        ALL TEXT
Layers:                   MEMORY-BASED
Keep Empty Checks:        NO

    --- SUMMARY
    ---
TOTAL CPU Time:            486|
TOTAL REAL Time:          488
TOTAL Original Layer Geometries: 1848129 (7319690)
TOTAL DRC RuleChecks Executed: 2334
TOTAL DRC Results Generated: 63276 (5625486)

```

Figure 5. DRC report from Calibre using DRC checks from TSMC16nm PDK. Most of the results from the DRC results were not passing the guidelines set by our PDK.

```

#####
# Generated by:      Cadence Innovus 23.14-s088_1
# OS:                Linux x86_64(Host ID ecs03.poly.edu)
# Generated on:      Fri Dec 12 19:52:56 2025
# Design:            system_top
# Command:           check_connectivity -type all -geometry_connect
-out_file ../files/system_top.conn.rpt -error 100000 -warning 50
#####
Verify Connectivity Report is created on Fri Dec 12 19:52:56 2025

Begin Summary
  3 Problem(s) (IMPVFC-96): Terminal(s) are not connected.
  507 Problem(s) (IMPVFC-200): Special Wires: Pieces of the net
are not connected together.
  505 Problem(s) (IMPVFC-92): Pieces of the net are not connected
together.
  1015 total info(s) created.
End Summary

```

Figure 6. LVS (connectivity) report through Innovus, checking for all connectivity errors. The results show 1015 errors with the majority containing ‘pieces of the net are not connected together’ errors. Although not seen in this screenshot, this is a large improvement from the 10000+ connectivity error we began with.

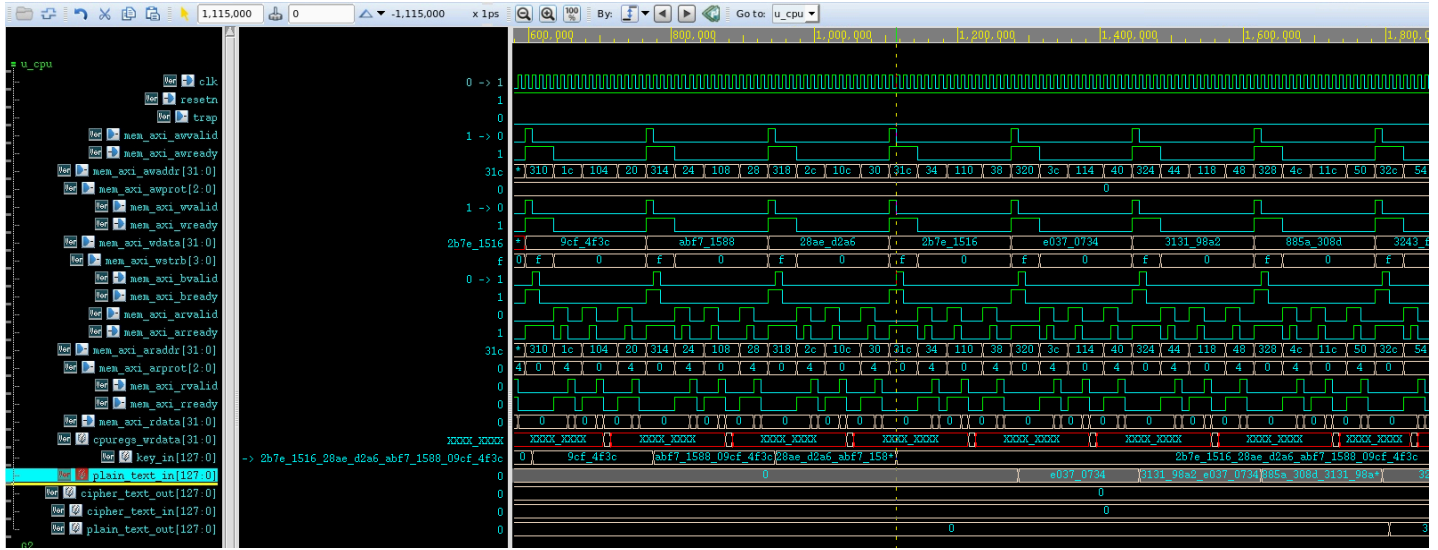


Figure 7. The waveform shows how the key is being written into our AES registers from the CPU. Because our key is 128bits but our AXI module is only 32 bits, we must perform four store operations to write all 128 bits of the key. The first is written to address 0x310 (time 600,000), after our main module (CPU) asserts address/data is valid while the sub module (AES) asserts it is ready to receive the address/data. The next address is 0x314, followed by 0x318 and finally ending at 0x31c. We can see key_in incrementally filling in by 32 bits on each of those intervals.

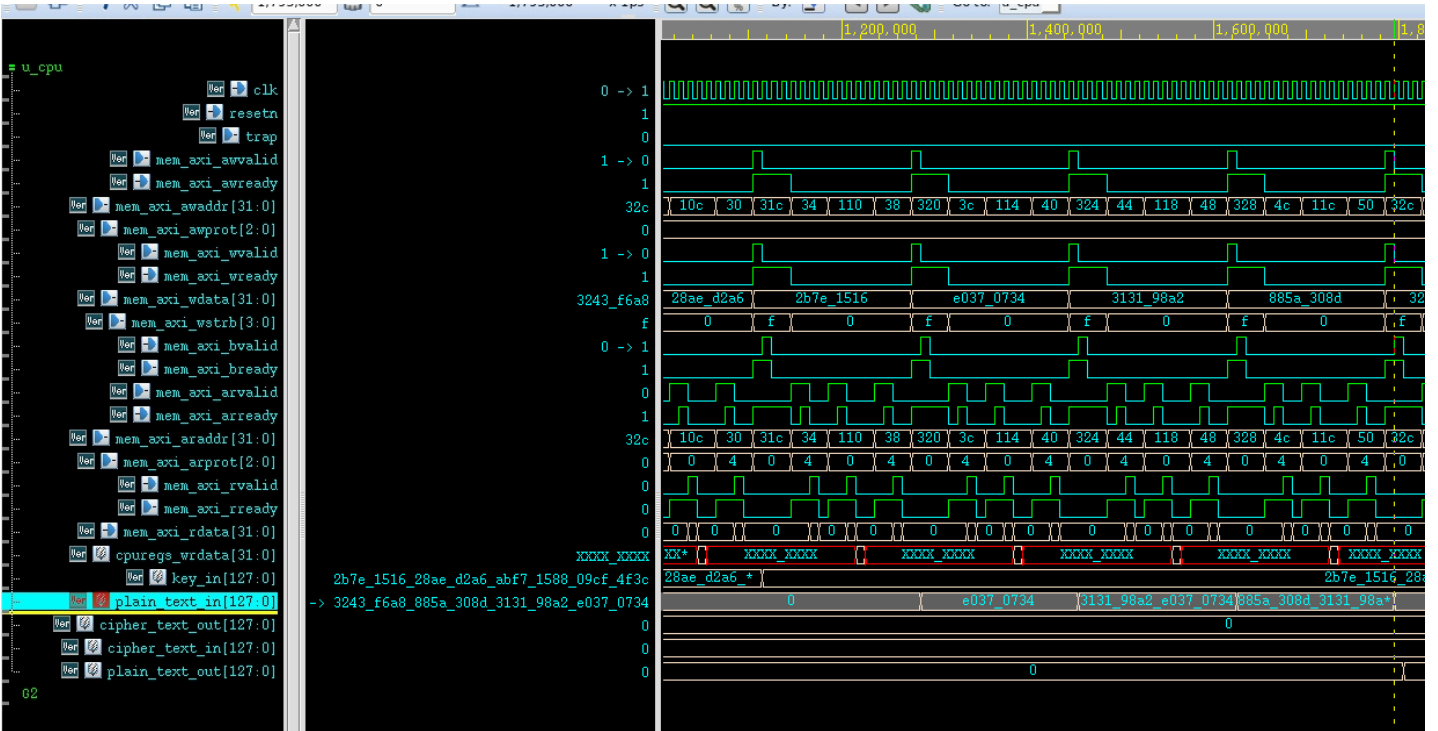
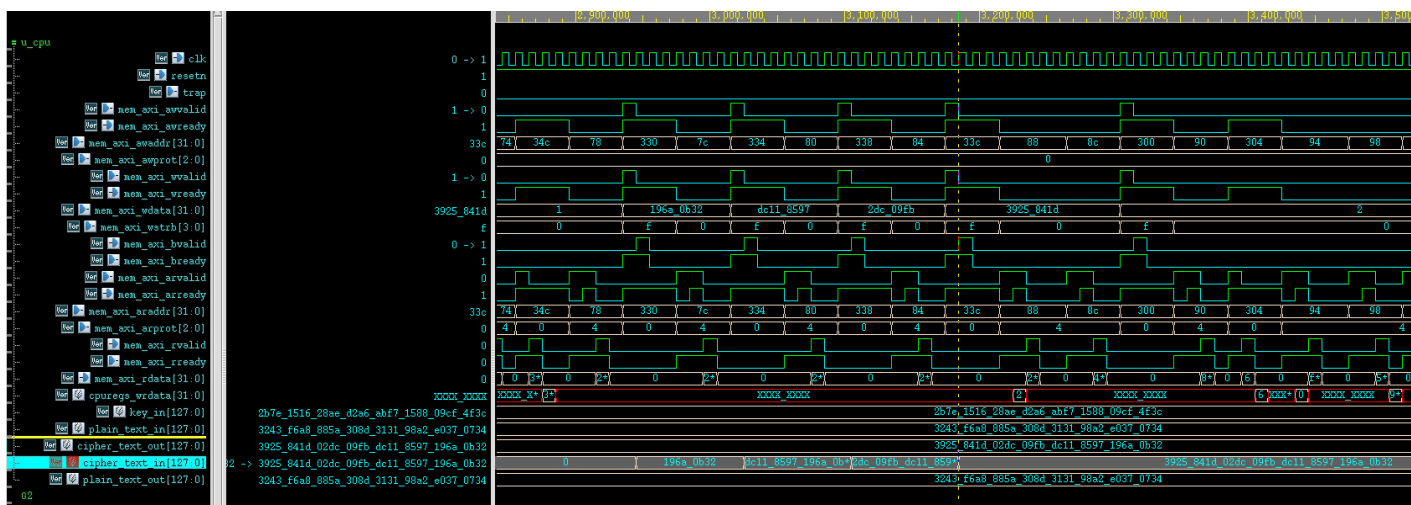
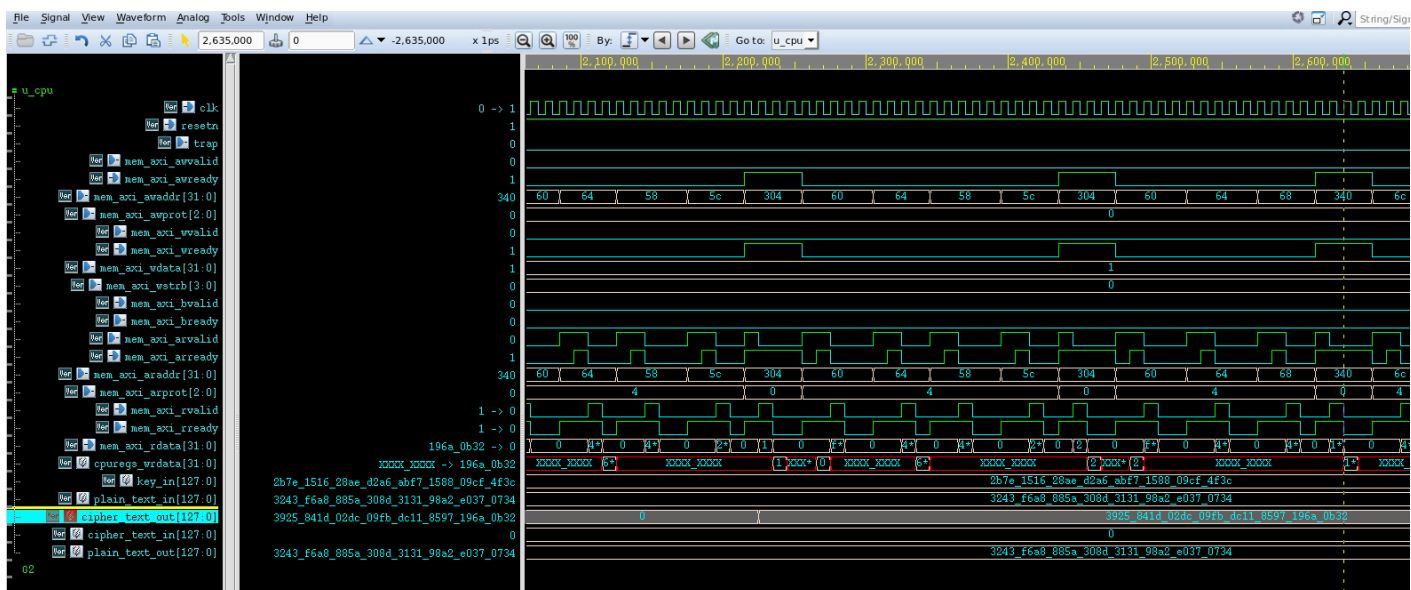
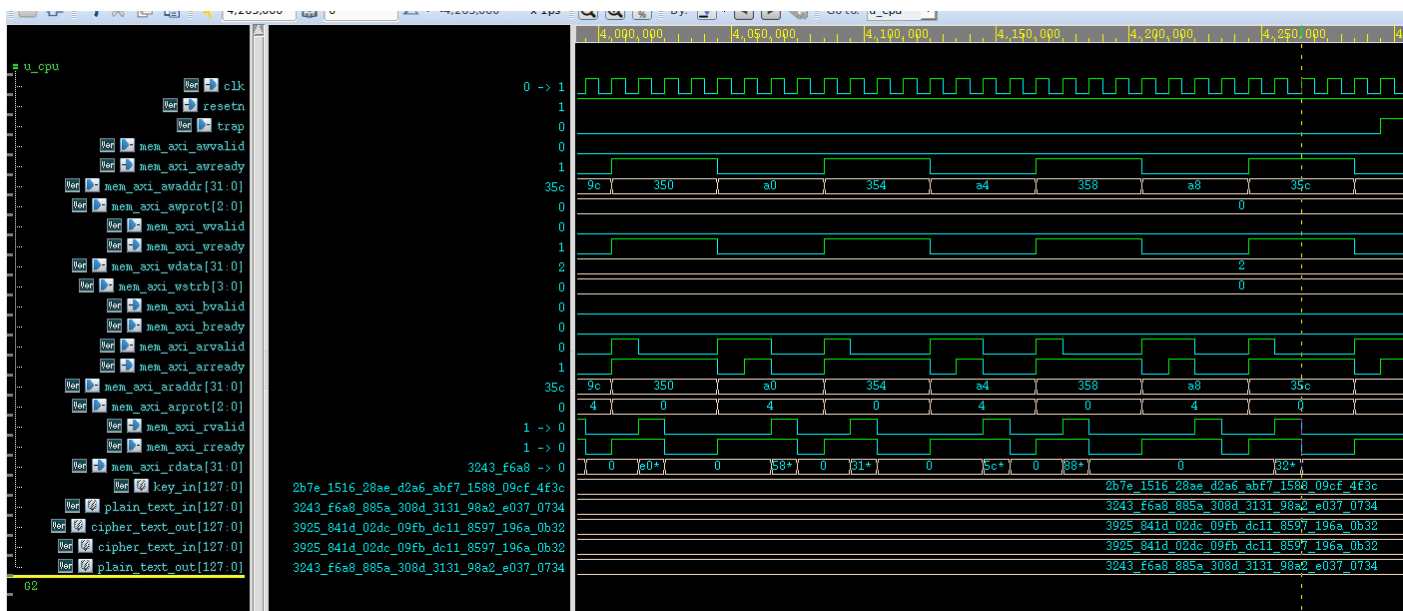
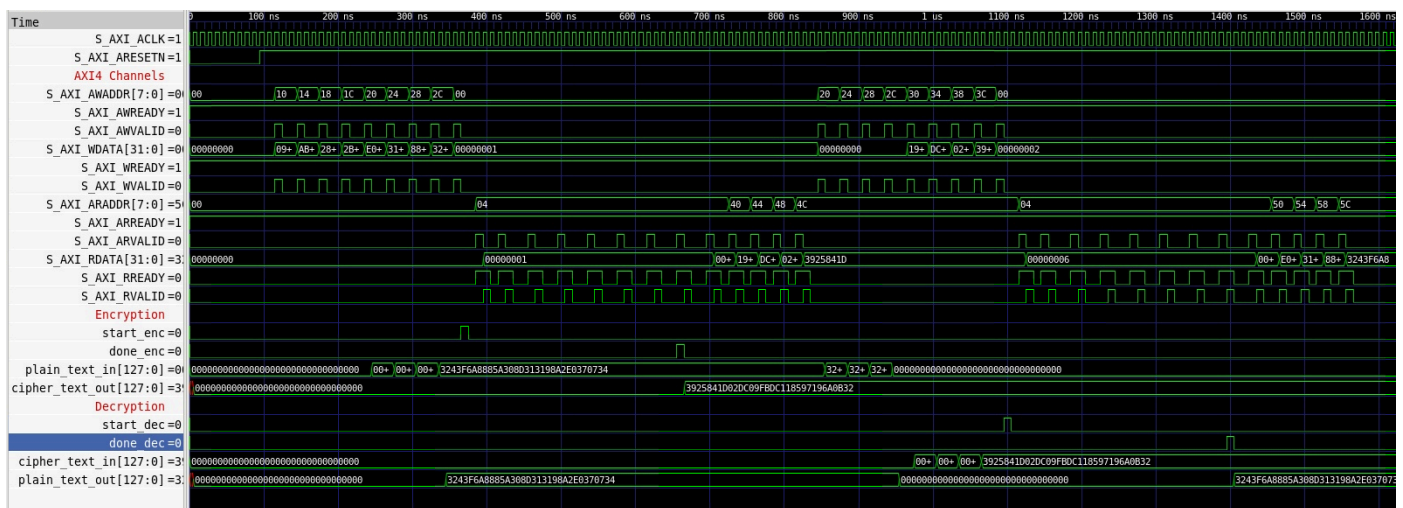


Figure 8. Depiction of the first half of encryption in which we write the plain text from the CPU into AES registers, following a similar procedure to writing key_in. We note the AES registers holding the plain_text are located at address 320, 324, 328 and 32c. Around 1,800,000ns all 128 bits of our plain_text are entered and encryption is ready to start.





Waveform Simulations of AES for Functional Verification



Additional Work

```

30 set_db information_level 7
31 set_db init_lib_search_path ${LIB_PATH}
32 set_db init_hdl_search_path ${RTL_PATH}
33 set_db library $LIBRARY
34 read_libs $LIBRARY
35
36 read_hdl $FILE_LIST
37 elaborate $DESIGN
38 check_design $DESIGN -unresolved
39
40 read_sdc $SDC_FILE
41
42
43 if {[file exists ${LOG_PATH}]} {
44     file mkdir ${LOG_PATH}
45     puts "Creating directory ${LOG_PATH}"
46 }
47 if {[file exists ${OUTPUTS_PATH}]} {
48     file mkdir ${OUTPUTS_PATH}
49     puts "Creating directory ${OUTPUTS_PATH}"
50 }
51 if {[file exists ${REPORTS_PATH}]} {
52     file mkdir ${REPORTS_PATH}
53     puts "Creating directory ${REPORTS_PATH}"
54 }
55
56 set_db syn_generic_effort $SYN_EFFORT
57 set_db syn_map_effort $MAP_EFFORT
58 set_db syn_opt_effort $OPT_EFFORT
59
60 syn_generic
61 syn_map
62 syn_opt
63
64 report_area $DESIGN > ${REPORTS_PATH}/${DESIGN}.area.rpt
65 report_datapath $DESIGN > ${REPORTS_PATH}/${DESIGN}.datapath_incr.rpt
66 report_gates $DESIGN > ${REPORTS_PATH}/${DESIGN}.gates.rpt
67 report_timing > ${REPORTS_PATH}/${DESIGN}.timing.rpt
68 report_power > ${REPORTS_PATH}/${DESIGN}.power.rpt
69
70 write_hdl > ${OUTPUTS_PATH}/${DESIGN}.v
71 write_sdc > ${OUTPUTS_PATH}/${DESIGN}.sdc
72 write_sdf -timescale ns -nonnegchecks -recmem split -edges check_edge

```

```

set PDK_DIR                "/ip/tsmc/tsmc16adfp/stdcell"
set PDK_DIR2                "/ip/tsmc/tsmc16adfp/stdio"
set PDK_DIR3                "/ip/tsmc/tsmc16adfp/sram"
set ADDITIONAL_SEARCH_PATH "$PDK_DIR/NLDM $PDK_DIR2/NLDM $PDK_DIR3/NLDM"
script files.

set TARGET_LIBRARY_FILES    "N16ADFP_StdCell.ttt0p8v25c.db N16ADFP_StdIO.ttt0p8v25c.db N16ADFP_SRAM.ttt0p8v25c.db"

set NDM_DESIGN_LIB          "TOP.dlib"                ;# User-defined NDM
set NDM_REFERENCE_LIBS      "N16ADFP_StdCell_physicalonly.ndm reflib.ndm N:"

#####
# Logical Library Settings
#####
set_app_var search_path     "$search_path $ADDITIONAL_SEARCH_PATH"
set_app_var target_library  $TARGET_LIBRARY_FILES
set_app_var link_library    "*" $target_library"

#####
# Physical Library Settings
#####

define_design_lib WORK -path ./work                ;# Location of "analyze"d files

# .....
# Verify Settings
# .....

echo "\n===== Library Settings ====="
echo "search path:      $search_path"
echo "link library:      $link_library"
echo "target library:     $target_library"
echo "physical libraries: $NDM_REFERENCE_LIBS"
echo "physical design library: $NDM_DESIGN_LIB"
echo "\n===== "

```

```

analyze -format sverilog {fsm4_pkg.sv mux.sv
add_roundkey.sv dff.sv key_expansion.sv mix_cols.sv
s_box.sv s_box_inv.sv shift_rows.sv sub_bytes.sv
sub_bytes_inv.sv pipeline_reg.sv decryption_fsm.sv
decryption.sv encryption_fsm.sv encryption.sv
aes_core.sv top_aes.sv sram_axi_wrapper.v picorv32.v
system_top.v}

elaborate system_top

list_libs
list_designs

current_design

read_sdc constraints/design_constraints.sdc

compile_ultra

report_timing

write -format verilog -hierarchy -output synthesis/
files/system_top.v

write_sdc synthesis/files/post_syn.sdc

```

Figure 17. Scripts for genus(left), and design compiler(middle and right). By creating scripts we were able to generate netlists without using the UI for their respective tools, thus decreasing the time it took to process any RTL changes.

```

set DATE [clock format [clock seconds] -format "%b%d %H:%M:%S"]
set OUTPUT_DIR "physical_design/logs/output_${DATE}"

puts "INFO: Loading Libraries..."

set_db design_process_node 16
set init_lef_file {/ip/tsmc/tsmc16adfp/tech/APR/
N16ADFP_APR_Innovus/N16ADFP_APR_Innovus_11M.10a.tlef
/ip/tsmc/tsmc16adfp/pll/LEF/lef/
n16adfp_pll_100a_10lm.lef /ip/tsmc/tsmc16adfp/stdcell/lef/lef/N16ADFP_StdCell.lef /ip/tsmc/tsmc16adfp/sram/lef/lef/N16ADFP_SRAM_100a.lef /ip/tsmc/tsmc16adfp/stdio/lef/lef/N16ADFP_StdIO.lef}
# set init_original_verilog_files syn/outputs/
# picorv32_axi.v
set init_verilog syn/outputs/system_top.v
set init_top_cell "system_top"
set init_pwr_nets {"VDD" "VSS"}
# "VDD" "VSS" "VDD" "VSS" "AHVDD"
set init_gnd_nets {"VSS"}
# "VDD" "VSS" "VDD" "VSS" "AHVDD"
# "VDD" "VSS" "VDD" "VSS" "AHVDD"
# /ip/tsmc/tsmc16adfp/tech/APR/N16ADFP_APR_Innovus/
# N16ADFP_APR_Innovus_11M.10a.tlef /ip/tsmc/tsmc16adfp/pll/LEF/lef/n16adfp_pll_100a_10lm.lef /ip/tsmc/
# tsmc16adfp/sram/lef/lef/N16ADFP_SRAM_100a.lef /ip/tsmc/
# tsmc16adfp/stdcell/lef/lef/N16ADFP_StdCell.lef /ip/
# tsmc16adfp/stdio/lef/lef/N16ADFP_StdIO.lef
set init_lef_check_antenna 1

init_design

```

```

1 # routing & fillers if needed
2 set DATE [clock format [clock seconds] -format "%b%d %H:%M:%S"]
3 set REPORT_DIR "physical_design/logs/report_${DATE}"
4
5 # this script takes forever, so go get food or go to the gym while I
6
7 set_db route_detail_fix_antenna 1
8 set_db route_antenna_diode_insertion 0
9 set_db route_with_timing_driven 1
10 set_db route_with_eco 0
11 set_db route_with_litho_driven 0
12 set_db route_detail_post_route_litho_repair 0
13 set_db route_with_si_driven 1
14 set_db route_detail_auto_stop 0
15 set_db route_selected_net_only 0
16 set_db design_top_routing_layer 9
17 set_db design_bottom_routing_layer 1
18 set_db route_detail_end_iteration 10
19 set_db route_with_timing_driven true
20 set_db route_with_si_driven true
21 route_design -global_detail
22
23 set fillerCells {list FILL16BWP16P90, FILL16BWP20P90, FILL16BWP16P90,
24
25 opt_design -post_route -report_dir $REPORT_DIR
26 opt_design -post_route -hold -report_dir $REPORT_DIR
27 opt_design -post_route -hold -setup -report_dir $REPORT_DIR
28
29 set_db timing_analysis_type ocv
30 set_db timing_analysis_cprr both
31 time_design -post_route
32 time_design -post_route -hold
33
34 #getFillerMode -quiet
35 #add_fillers -base_cells $fillerCells -prefix FILLER

```

```

# verify DRC
set_db check_drc_area {0 0 0}
set_db check_drc_disable_rules {}
set_db check_drc_nsr_spacing auto
set_db check_drc_check_only default
set_db check_drc_inside_via def true
set_db check_drc_exclude_pg_net false
set_db check_drc_ignore_trial_route false
set_db check_drc_ignore_cell_blockage false
set_db check_drc_report $DRC_OUT
set_db check_drc_limit 1000
check_drc

# verify connectivity
check_connectivity -type all -geometry_connect -
out_file $CONN_OUT

### Save files
extract_rc
write_parasitics -rc_corner worst_case -spef_file
$WORSE_PARASITICS

extract_rc
write_parasitics -rc_corner best_case -spef_file
$BEST_PARASITICS

set_db timing_analysis_type ocv
set_db timing_analysis_cprr both
time_design -signoff
time_design -signoff -hold

write_stream $GDS_OUT -map_file /ip/tsmc/tsmc16adfp/
tech/APR/N16ADFP_APR_Innovus/
N16ADFP_APR_Innovus_Gdsout_11M.10a.map -lib_name
DesignLib -structure_name system_top -merge { /ip/tsmc/
tsmc16adfp/stdcell/GDS/N16ADFP_StdCell.gds /ip/tsmc/
tsmc16adfp/sram/GDS/N16ADFP_SRAM_100a.gds /ip/tsmc/
tsmc16adfp/stdio/GDS/N16ADFP_StdIO.gds /ip/tsmc/
tsmc16adfp/pll/GDS/n16adfp_pll_100a.gds } -
uniquify_cell_names -unit 1000 -mode all

```

Figure 18. TCL scripts for different steps in Innovus including setup (left), detailed routing (middle), and generating GDS file (right). Additional scripts for steps such as floorplanning, and placement were also built but not shown above. These scripts for Innovus allowed us to automate the physical design process, facilitating less time between design changes to testing for DRC, LVS, and STA.

```

SIM      ?= vcs
SIM_FLAGS ?= -full64 -sverilog -kdb -lca -debug_access+all -timescale=1ns/1ps

BUILD_DIR ?= build
FILELIST  ?= filelist.f

SIM_EXE := $(BUILD_DIR)/tb_system_top

.PHONY: all top clean help

all: top

# Build and run tb_system_top using the provided filelist
top: $(SIM_EXE)
    @echo "Running $(SIM_EXE)"
    $(SIM_EXE)

$(SIM_EXE): $(FILELIST) | $(BUILD_DIR)
    @echo "Compiling tb_system_top with vcs using $(FILELIST)"
    $(SIM) $(SIM_FLAGS) -o $@ -top tb_system_top -f $(FILELIST)

$(BUILD_DIR):
    mkdir -p $@

clean:
    rm -rf $(BUILD_DIR)

help:
    @echo "Make targets:"
    @echo "  all - Build and run tb_system_top (same as 'top')"
    @echo "  top - Build and run tb_system_top using filelist.f"
    @echo "  clean- Remove build outputs"
    @echo "  help - Show this help"
    @echo ""
    @echo "Variables:"
    @echo "  SIM      (default: vcs)"
    @echo "  SIM_FLAGS (default: -full64 -sverilog -kdb -lca -debug_access+all -timescale=1ns/1ps)"
    @echo "  BUILD_DIR (default: build)"
    @echo "  FILELIST  (default: filelist.f)"

```

```

SIM      ?= vcs
SIM_FLAGS ?= -full64 -sverilog -kdb -lca -debug_access+all -timescale=1ns/1ps

BUILD_DIR ?= build

PKG_SRCS := $(wildcard pkgs/*.sv)
# Filter out testbenches (tb/*.sv) from the RTL compile set.
RTL_SRCS := $(filter-out tb/%,$(filter-out tb/*.sv,$(wildcard *.sv)))

# Collect testbenches and ensure tb_top_aes is last.
TB_FILES_ALL := $(wildcard tb/tb_*.sv)
TB_FILES      := $(filter-out tb/tb_top_aes.sv,$(TB_FILES_ALL)) tb/tb_top_aes.sv
TB_TOPS       := $(notdir $(basename $(TB_FILES)))
TB_EXES       := $(addprefix $(BUILD_DIR)/,$(TB_TOPS))

.PHONY: all top clean help

# Build and run every testbench; tb_top_aes runs last.
all: $(TB_EXES)
    @set -e; \
    for tb in $(TB_TOPS); do \
        exe="$(BUILD_DIR)/$$tb"; \
        echo "Running $$tb from $$exe"; \
        "$$exe"; \
    done

# Build and run only tb_top_aes
top: $(BUILD_DIR)/tb_top_aes
    @echo "Running $(BUILD_DIR)/tb_top_aes"
    $(BUILD_DIR)/tb_top_aes

# Pattern rule to compile each testbench
$(BUILD_DIR)/%: tb/*.sv $(PKG_SRCS) $(RTL_SRCS) | $(BUILD_DIR)
    @echo "Compiling $* with vcs"
    $(SIM) $(SIM_FLAGS) -o $@ -top $* $(PKG_SRCS) $(RTL_SRCS) $<

$(BUILD_DIR):
    mkdir -p $@

clean:
    rm -rf $(BUILD_DIR)
    rm *.vcd
    rm *.fsdb
    rm *.log
    rm *.key

```

Figure 19. Shown above is a makefile which compiles all the verilog code instantiated by our top module (including CPU, SRAM, AES and AXI interconnect), and additionally builds and runs all our tests. Another makefile was created for testing AES functionality individually but not shown above. Through the makefile we were able to automate testing and verify the functionality of our modules quicker, reducing repetitive work.